Data Science and AI Lab Project

GROUP I

RAG-BASED LLM CODE REVIEW AGENT

MILESTONE 4

RAG-BASED CODE REVIEW AGENT

Milestone 4: Data Preparation, Preprocessing, and Chunking

Project Scope

Build a ground-truth dataset from historical Pull Requests, implement AST-aware code chunking, and systematically optimize RAG model inference.

Target Repositories

`django` `flask` `scikit-learn` `pandas` `fastapi`

Key Outcomes

- Clean, linter-verified dataset
- Reproducible RAG experiments
- Grounded, interpretable citations

Data Preparation — Evolution of Extraction

From fragmented, repository-specific scripts to a unified, scalable orchestration pipeline.

From Scripts to Pipeline

× Failed Attempt: Standalone scripts (e.g., `django_pr_extraction.py`) were unscalable, creating code duplication and inconsistent error handling.

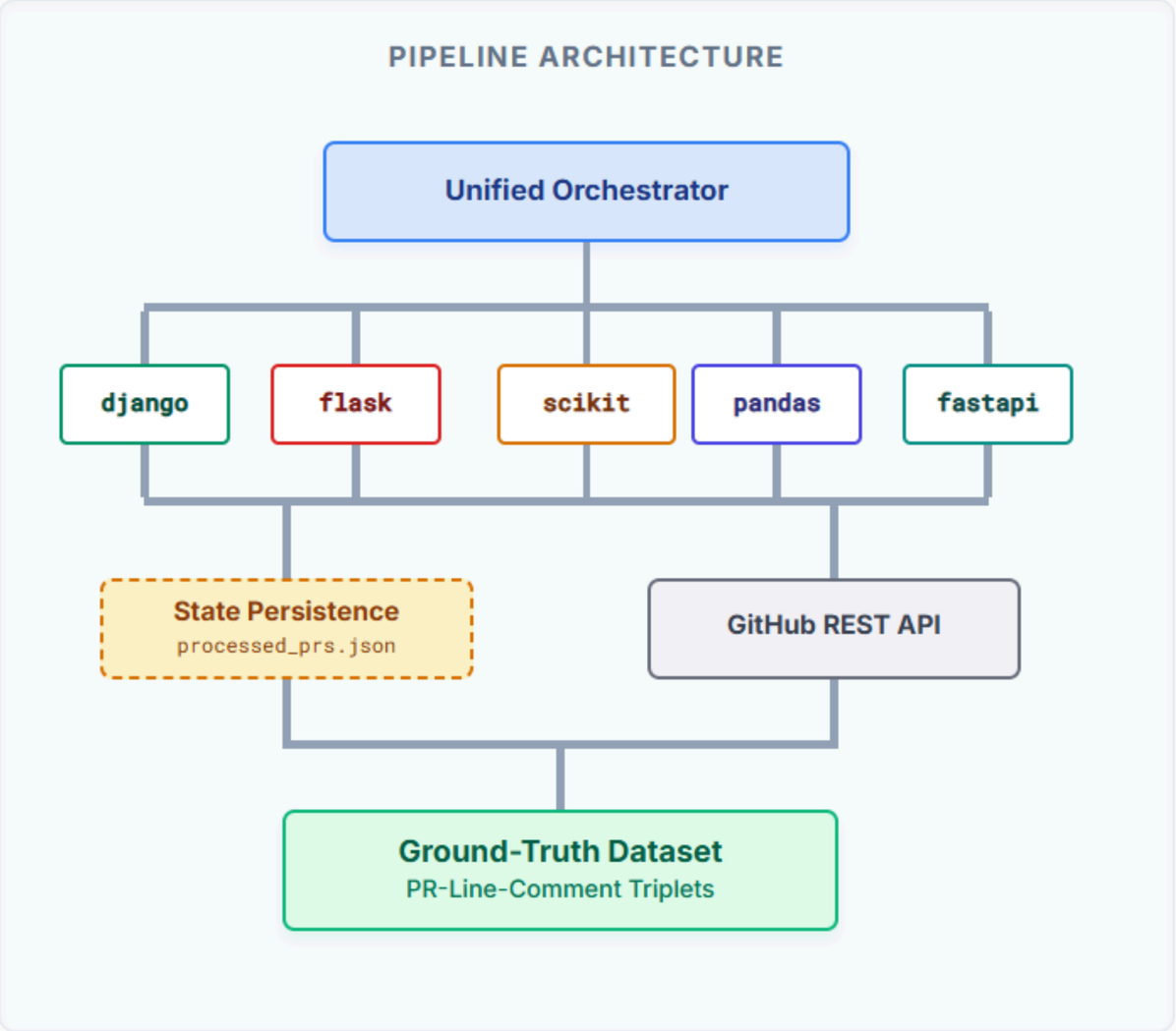
✓ Final Architecture: A unified, parameter-driven `pr_extraction.py` capable of handling any target repository via CLI arguments.

Core Capabilities

- **State Persistence:** Uses `processed_prs.json` to seamlessly resume extraction if interrupted by API rate limits.
- **Parallel Execution:** Orchestrated multiprocessing via `subprocess.Popen` across 5 diverse Python repositories concurrently.
- **Targeted Output:** Built a massive initial pool by successfully gathering ~150 specific PR-line-comment triplets per repository.

Impact

> Moving from "crawl and store" to a resilient pipeline guaranteed the **scalability, data consistency, and rate-limit safety** necessary to construct a high-quality machine learning dataset.



Filtering & Automated Labeling

Implementing a "Linter-in-the-Loop" strategy for deterministic noise reduction and classification.

Noise Reduction Heuristics

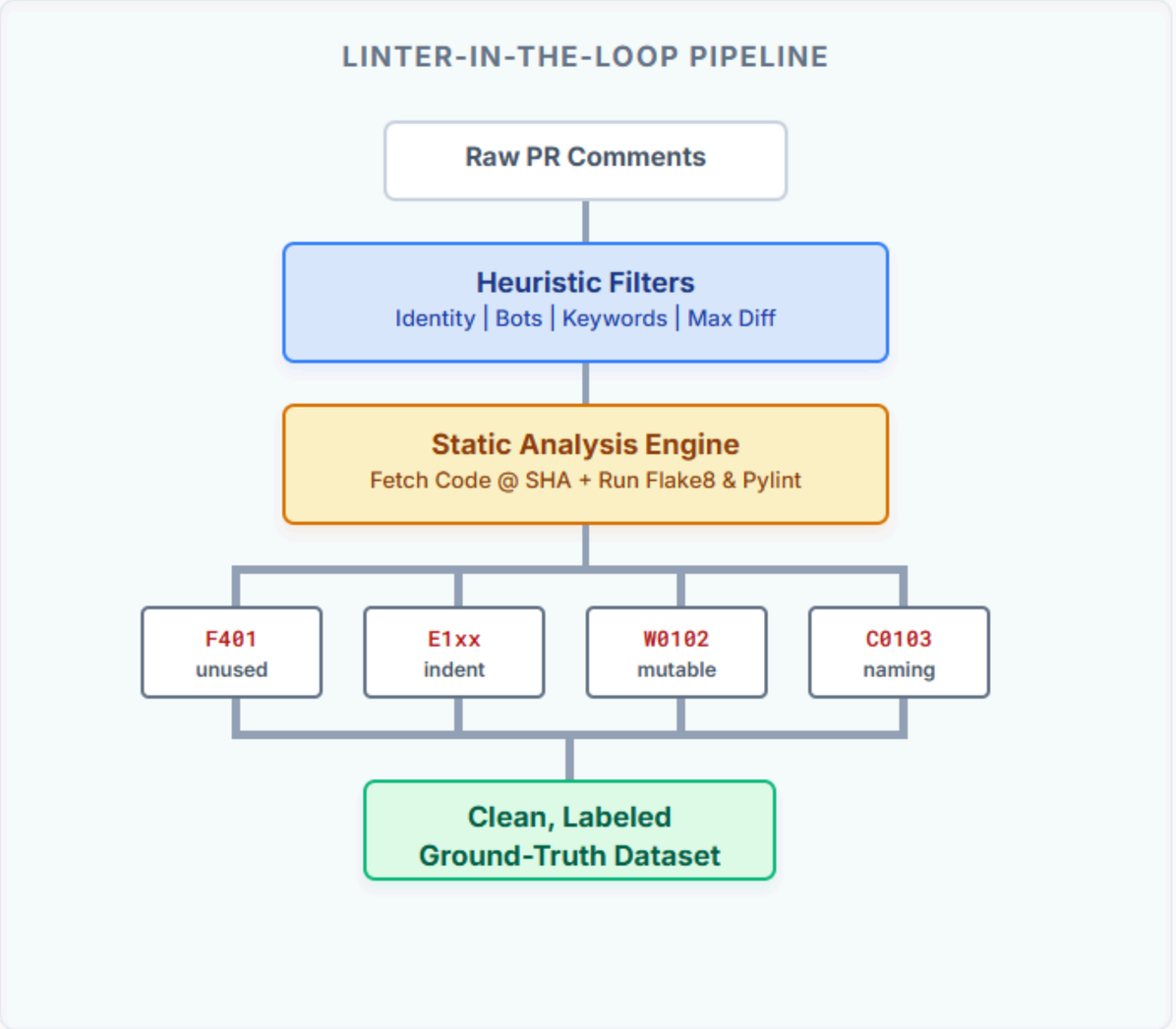
- **Identity Check:** `reviewer != author` prevents self-notes ("I will fix this later") from polluting the dataset.
- **Bot Exclusion:** Regex filters out repetitive CI/CD reports (e.g., `*bot*`, `codecov`).
- **Signal Keywords:** Prioritized comments containing correctional signals: `should, avoid, naming, unused, fix, mutable`.
- **Diff Constraints:** Discarded PRs with >200 changed lines, focusing learning on targeted, style-centric PRs.

Deterministic Labeling (Static Analysis)

We replaced manual/LLM labeling with deterministic static analysis to eliminate hallucination and high API costs at scale.

Flake8 & Pylint Code Mapping:

`F401 (Flake8)`	→	**unused_import**
`E1xx (Flake8)`	→	**indentation**
`W0102 (Pylint)`	→	**mutable_default**
`C0103 (Pylint)`	→	**naming_convention**



Chunking Strategies — From Lines to AST

Evolving from naive line-based windows to structure-aware semantic blocks.

× Failed Experimental Strategies

Strategy A: Fixed-Size Sliding Window

Thought Process: Take the violation line and include exactly 10 lines above and below.

> **Failure Mode:** Often cut off the top of a function (`def` line) or class start. Without the signature, the LLM cannot verify naming conventions or mutable defaults.

Strategy B: Unified Patch Only

Thought Process: Only provide the `+` and `-` lines from the git diff.

> **Failure Mode:** Complete lack of semantic context. The LLM has no idea which class the modified method belongs to.

VS

✓ Final: AST-Aware Chunking

Tree-sitter AST Pipeline

1. **Parse:** Generate a full concrete syntax tree of the Python file.
2. **Traverse:** Use DFS to find the deepest node that fully encompasses the violation line.
3. **Expand:** Automatically grab the entire `function_definition` or `class_definition` node to provide full surrounding context.

System Resiliency & Outcome

Fallback Strategy: If syntax errors prevent AST building (common in older PRs), system reverts to 10-line window to ensure zero data loss.

> **Final Result:** Every dataset sample contains a **semantically complete code block**, giving the LLM identical information to a human reviewer.

RAG Architecture & Inference Setup

Combining FAISS dense retrieval with Groq-powered LLMs for structured code review generation.

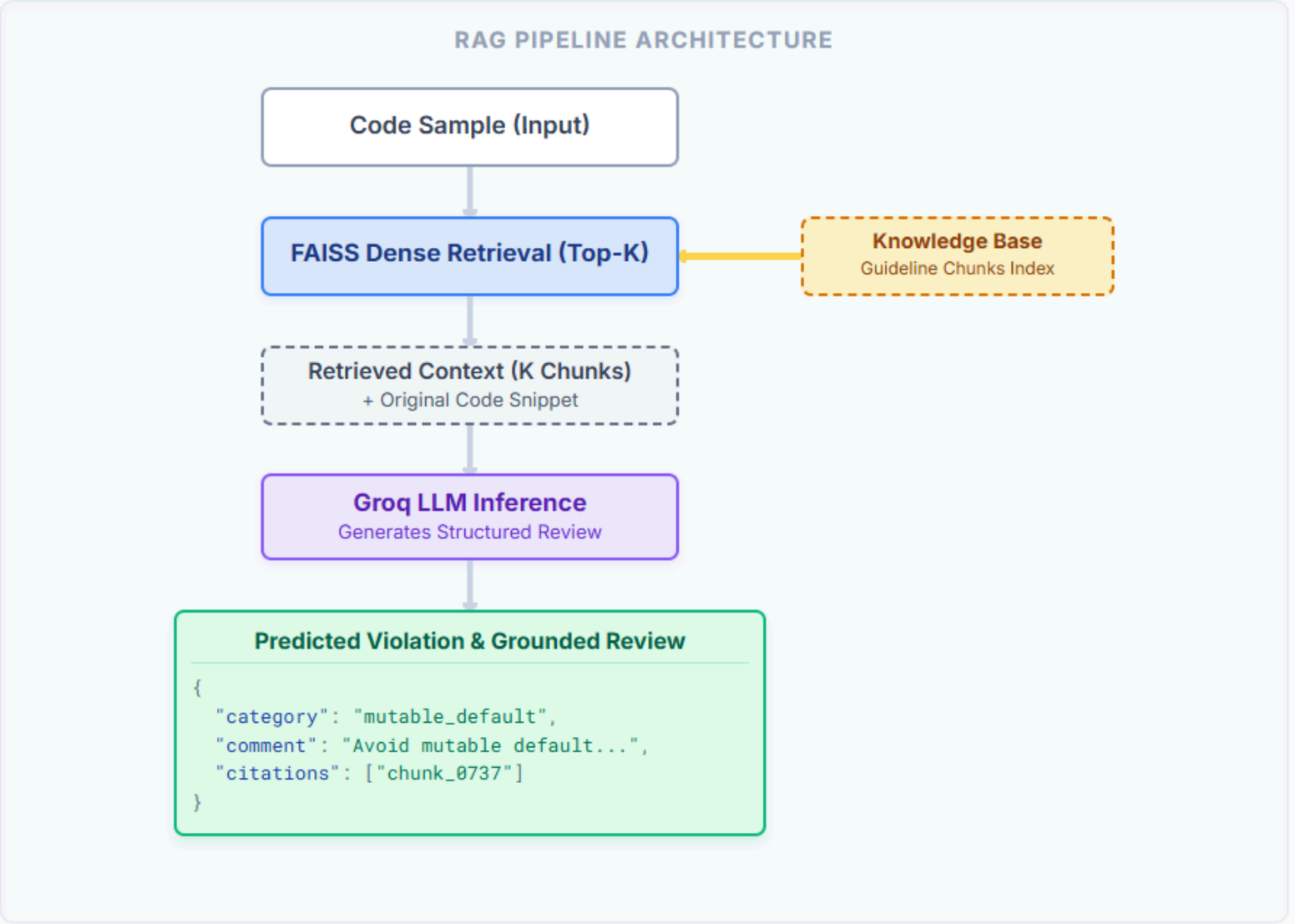
Dataset & Knowledge Base

- **1,235 labeled code samples** spanning 5 core violation classes (including `documentation_formatting`).
- **1,000+ guideline chunks** derived from PEP8 and project-specific guides. Rule-focused chunks limited to ~200–400 tokens.
- Metadata integration: Each chunk mapped with `rule_id` and source.

Configuration & Operations

Model Engine:	`gpt-oss-20b` (Groq API)
Rate Limits:	30 RPM, 2.0s min interval
Resilience:	Max 3 retries, fixed random seeds
Input Tokens:	Up to ~2,000 token context window

All API configurations are serialized for deterministic reproducibility across hyperparameter experiments.



RAG Model Inference & Hyperparameter Experiments

Systematic exploration of temperature and Top-K retrieval on review generation accuracy.

Overall Accuracy by Cohort & Configuration

Cohort	Best Configuration	Peak Accuracy	Key Finding
V1 (n=12)	Temp 0.1 | K=7	**0.4167**	Low temp reduces hallucination; high K provides context.
V2 (n=35)	Temp 0.3 | K=1	**0.3714**	Smaller K reduces noise; slight temp boost adds diversity.

JSON Parse Quality & Robustness

V1 VALID RATE	V2 VALID RATE
78%	91%

+13% improvement driven by refined prompt engineering, larger context windows, and model familiarity with structured outputs.

Per-Class Performance (V2, Temp=0.1, K=1)

Violation Class	Precision	Recall	F1-Score
`unused_import`	0.27	0.57	0.36
`naming_convention`	0.50	0.29	0.36
`mutable_default`	0.50	0.14	0.22
`documentation_formatting`	0.30	0.43	0.35
`indentation`	0.25	0.14	0.18

Key Observations & Trade-offs

- **U-Shaped Context Trade-off:** Very few retrieved chunks (K=1) may miss context, but too many (K=7) introduce conflicting guideline noise.
- **Dataset Size Influence:** V2's larger, more diverse cohort benefits from the higher temperature (0.3) to enhance reasoning diversity.
- **Class-Specific Strengths:** RAG excels at explicit conventions (Precision 0.50 for `naming` and `mutable_default`), but struggles with nuanced syntax like indentation.

All results are reproducible with fixed random seeds and serialized API configurations.

Results, Observations & Bottlenecks

Comprehensive evaluation of the RAG inference pipeline across varied hyperparameter configurations.

QUANTITATIVE PERFORMANCE SUMMARY

Average Accuracy: V1 0.3214 → V2 0.3143
Stability: -2.22% (slight decrease indicates dataset diversity challenge)

V1 Champion: Temp 0.1 + K=7 — **Accuracy: 0.4167**
V2 Champion: Temp 0.3 + K=1 — **Accuracy: 0.3714**

✓ What Worked Well

- **FAISS Integration**
  Semantic search successfully retrieved relevant guidelines for syntax-heavy violations.
- **Groq API Stability**
  Rate limiting and retry logic maintained consistent inference without timeouts.
- **Parse Quality at Scale**
  V2 achieved 94% valid JSON parse rate, demonstrating model reliability.
- **Interpretability**
  Retrieved chunk citations allow validation of the model's reasoning process.

⚠ What Underperformed

- **Nuanced Violations**
  Classes like `indentation` and `docs` require broader context beyond keyword matching.
- **Knowledge Base Coverage**
  Corner-case violations (e.g., unconventional naming in specific domains) lacked relevant chunks.
- **Context Sensitivity**
  Large K values (K=7) diluted the signal-to-noise ratio, introducing contradictory guidance.

× System Bottlenecks

- **Retrieval Precision Limits**
  FAISS relies strictly on embedding similarity; if code context is ambiguous, retrieval fails.
- **API Rate Limiting**
  The 30 RPM cap severely limited exploration speed, requiring careful request scheduling.
- **Dataset Imbalance**
  The V2 cohort had only 7 samples per violation class, leading to high variance in per-class metrics.

Insight: A U-shaped trade-off curve balances context retrieval (K) and model creativity (Temperature).

Conclusion & Next Steps

Establishing a robust production-ready foundation and mapping the path forward.

★ Production-Ready Foundation

✓ **94% Valid JSON Parse Quality**
Demonstrated strong model reliability for generating strictly structured outputs at scale.

✓ **Grounded, Interpretable Reviews**
Integration of direct chunk citations allows human validation of the LLM's reasoning process.

✓ **Established Accuracy Baseline**
Set a reproducible starting point (0.31–0.37) for hyperparameter and architectural tuning.

📦 Reproducible Artifacts

- **FAISS Index & Corpus:** Pre-built dense embeddings of chunked guidelines.
- **Prompt Templates:** Standardized system & user prompts for structured reviews.
- **Results & Configs:** Serialized API credentials and raw predictions for all experiments.

🚀 Roadmap for Improvement

1. **Hybrid Retrieval Architecture**
Combine dense retrieval (FAISS) with sparse lexical search (BM25) to improve recall on rare, keyword-specific violations.

2. **Expand Evaluation Dataset**
Scale testing from 35 to 100–200 samples to stabilize per-class performance metrics and significantly reduce variance.

3. **Fine-Tuned Embeddings**
Train domain-specific embedding models directly on matched code + guideline pairs to dramatically improve semantic alignment.

4. **Few-Shot Prompting**
Introduce curated in-context examples to guide the model toward more systematic, deductive reasoning prior to classification.

5. **Multi-Stage Classification Cascade**
Deploy cascading classifiers that first predict a broad violation category, then refine to a specific type via specialized sub-classifiers.

"Validating that a well-designed RAG pipeline can learn from structured knowledge—the critical foundation for building autonomous code review agents."